

Funções em Linguagem C

1. Como declarar uma função

Em C, uma função deve ser declarada antes de ser utilizada. A declaração de uma função envolve especificar seu tipo de retorno, o nome da função e os parâmetros que ela recebe. A seguir, um exemplo simples:

```
1  #include <stdio.h>
2
3  // Declaração da função (protótipo)
4  int soma(int a, int b);
5
6  int main() {
7      int resultado = soma(5, 3);
8      printf("Resultado: %d\n", resultado);
9      return 0;
10 }
11
12 // Definição da função
13 int soma(int a, int b) {
14     return a + b;
15 }
```

No exemplo acima, a função `soma` recebe dois inteiros (`int a` e `int b`) e retorna a soma deles (`return a + b;`). A declaração da função é feita antes do `main` com `int soma(int a, int b);`.

2. O que é um protótipo de uma função

O protótipo de uma função é uma declaração que informa ao compilador sobre a função, sem necessariamente fornecer a implementação completa. Isso permite que a função seja usada antes de sua definição no código. O protótipo inclui o tipo de retorno, o nome da função e os tipos dos parâmetros.

No exemplo anterior:

```
4  int soma(int a, int b);
```

Este é o protótipo da função `soma`. Ele informa ao compilador que existe uma função chamada `soma` que retorna um `int` e recebe dois parâmetros `int`.

3. Passagem de valores para uma função

Existem duas maneiras principais de passar valores para uma função em C: por valor e por referência.

3.1 Por Valor

Quando os argumentos são passados por valor, a função recebe uma cópia dos valores passados, ou seja, qualquer alteração feita nos parâmetros dentro da função não afeta as variáveis originais.

Exemplo:

```
1  #include <stdio.h>
2
3  void incrementa_por_valor(int x) {
4      x = x + 1;
5      printf("Dentro da função: %d\n", x);
6  }
7
8  int main() {
9      int a = 5;
10     incrementa_por_valor(a);
11     printf("Depois da função: %d\n", a);
12     return 0;
13 }
```

Saída:

```
1  Dentro da função: 6
2  Depois da função: 5
```

3.2 Por Referência

Quando os argumentos são passados por referência, a função recebe o endereço da variável original, permitindo que qualquer alteração feita no parâmetro dentro da função afete a variável original.

Exemplo:

```
1  #include <stdio.h>
2
3  void incrementa_por_referencia(int *x) {
4      *x = *x + 1;
5      printf("Dentro da função: %d\n", *x);
6  }
7
8  int main() {
9      int a = 5;
10     incrementa_por_referencia(&a);
11     printf("Depois da função: %d\n", a);
12     return 0;
13 }
```

Saída:

```
1  Dentro da função: 6
2  Depois da função: 6
```

4. Arquivos externos .h para funções

Em programas maiores, é comum organizar o código separando a declaração de funções em arquivos de cabeçalho (.h) e suas definições em arquivos de implementação (.c).

Exemplo de uso de um arquivo .h:

math_operations.h:

```
1  #ifndef MATH_OPERATIONS_H
2  #define MATH_OPERATIONS_H
3
```

```

4 int soma(int a, int b);
5 int subtracao(int a, int b);
6
7 #endif

```

math_operations.c:

```

1 #include "math_operations.h"
2
3 int soma(int a, int b) {
4     return a + b;
5 }
6
7 int subtracao(int a, int b) {
8     return a - b;
9 }

```

main.c:

```

1 #include <stdio.h>
2 #include "math_operations.h"
3
4 int main() {
5     int a = 5, b = 3;
6     printf("Soma: %d\n", soma(a, b));
7     printf("Subtração: %d\n", subtracao(a, b));
8     return 0;
9 }

```

No exemplo, o arquivo `math_operations.h` contém os protótipos das funções, `math_operations.c` contém as definições das funções, e `main.c` usa essas funções.

Para lembrar:

- **Declaração de uma função:** Define o tipo de retorno, o nome da função e os parâmetros.
- **Protótipo de uma função:** Informa ao compilador sobre uma função antes de sua implementação.
- **Passagem por valor:** Envia uma cópia do valor para a função.
- **Passagem por referência:** Envia o endereço da variável, permitindo modificações na variável original.
- **Arquivos .h:** Usados para declarar funções e facilitar a organização do código em projetos maiores.

5. O uso de < > e “ ”

A diferença entre usar `<stdio.h>` com colchetes angulares (< >) e `"math_operations.h"` com aspas duplas (" ") nas diretivas `#include` está relacionada ao local onde o compilador procura os arquivos de cabeçalho.

#include <stdio.h>

- **Colchetes angulares (< >)** indicam ao compilador que o arquivo de cabeçalho está localizado em um diretório padrão de sistema ou em um diretório de bibliotecas padrão. Isso geralmente significa que o compilador vai procurar o

arquivo nas pastas onde estão os arquivos de cabeçalho das bibliotecas padrão do C (como `stdio.h`, `stdlib.h`, etc.).

- Exemplo: `#include <stdio.h>` procura o arquivo `stdio.h` na lista de diretórios padrão que o compilador já conhece.

2. `#include "math_operations.h"`

- **Aspas duplas (" ")** indicam ao compilador que ele deve primeiro procurar o arquivo de cabeçalho no diretório onde o arquivo fonte atual está localizado. Se o compilador não encontrar o arquivo nesse diretório, ele pode então procurar nos diretórios padrão de sistema.
- Exemplo: `#include "math_operations.h"` diz ao compilador para procurar `math_operations.h` no mesmo diretório que o arquivo `.c` que o está incluindo. Se o arquivo não for encontrado ali, ele poderá procurar em outros diretórios, mas isso depende das configurações do compilador.

Para lembrar:

- **`#include <arquivo.h>`**: Usado para incluir arquivos de cabeçalho da biblioteca padrão do C ou de outros locais de bibliotecas configuradas globalmente no sistema.
- **`#include "arquivo.h"`**: Usado para incluir arquivos de cabeçalho que estão no diretório local ou em outros diretórios específicos do projeto, geralmente definidos pelo usuário.

5. Ponteiro para função

Um **ponteiro para função** em C é uma variável que armazena o endereço de uma função. Isso permite que você chame a função indiretamente através do ponteiro. Ponteiros para funções são usados quando você precisa passar funções como argumentos para outras funções, implementar callbacks, ou criar tabelas de funções, entre outros usos avançados.

Conceito Básico

Assim como você pode ter ponteiros para variáveis (por exemplo, inteiros, floats, etc.), você pode ter ponteiros que apontam para funções. Um ponteiro para função armazena o endereço de uma função, permitindo que você chame essa função através do ponteiro.

Como Declarar e Usar um Ponteiro para Função

1. Declarando um Ponteiro para Função

A sintaxe para declarar um ponteiro para função é:

```
tipo_de_retorno (*nome_do_ponteiro)(tipos_dos_parametros);
```

- **tipo_de_retorno**: Tipo de dado que a função retorna (por exemplo, `int`, `void`, etc.).
- **(*nome_do_ponteiro)**: Declaração do ponteiro para função.

- `tipos_dos_parametros`: Tipos dos parâmetros que a função aceita.

Exemplo:

Função simples soma:

```
1  #include <stdio.h>
2
3  // Função simples que soma dois inteiros
4  int soma(int a, int b) {
5      return a + b;
6  }
7
8  int main() {
9      // Declarando um ponteiro para função que aceita dois inteiros
        e retorna um inteiro
10     int (*ponteiro_para_funcao)(int, int);
11
12     // Atribuindo o endereço da função 'soma' ao ponteiro
13     ponteiro_para_funcao = &soma;
14
15     // Chamando a função através do ponteiro
16     int resultado = ponteiro_para_funcao(10, 20);
17
18     printf("Resultado: %d\n", resultado); // Saída: Resultado: 30
19
20     return 0;
21 }
```

Explicação:

- **Declaração do Ponto para Função:**

```
10 int (*ponteiro_para_funcao)(int, int);
```

Aqui, `ponteiro_para_funcao` é um ponteiro para uma função que retorna um `int` e aceita dois parâmetros do tipo `int`.

- **Atribuição do Endereço da Função:**

```
13 ponteiro_para_funcao = &soma;
```

O ponteiro `ponteiro_para_funcao` é atribuído ao endereço da função `soma`. O operador `&` é opcional; você poderia simplesmente escrever `ponteiro_para_funcao = soma;`.

- **Chamada da Função Através do Ponto:**

- ```
16 int resultado = ponteiro_para_funcao(10, 20);
```

A função `soma` é chamada indiretamente através do ponteiro `ponteiro_para_funcao`, passando os parâmetros `10` e `20`.

### Aplicações de Ponteiros para Funções

1. **Passagem de Funções como Argumentos:** Ponteiros para funções permitem que você passe funções como argumentos para outras funções, facilitando a implementação de callbacks e outras formas de programação modular.

2. **Tabelas de Funções:** Você pode criar arrays de ponteiros para funções, úteis para implementar tabelas de funções (como menus de comando).

#### Exemplo de Array de Ponteiros para Funções:

```
1 #include <stdio.h>
2
3 int soma(int a, int b) { return a + b; }
4 int subtracao(int a, int b) { return a - b; }
5
6 int main() {
7 // Array de ponteiros para funções
8 int (*operacoes[2])(int, int) = {soma, subtracao};
9
10 int x = 10, y = 5;
11
12 printf("Soma: %d\n", operacoes[0](x, y)); // Soma: 15
13 printf("Subtração: %d\n", operacoes[1](x, y)); // Subtração: 5
14
15 return 0;
16 }
```

Aqui, `operacoes[0]` aponta para a função `soma` e `operacoes[1]` para `subtracao`, permitindo a seleção dinâmica de funções.

#### Para lembrar:

- Um ponteiro para função é uma variável que armazena o endereço de uma função.
- Você pode usá-lo para chamar a função indiretamente.
- Ponteiros para funções são usados para passar funções como argumentos, criar callbacks, ou implementar tabelas de funções.

## 6. Exercícios

### 1: Nível Fácil

**Descrição:** Crie um programa em C que contenha uma função chamada `soma` que receba dois números inteiros como parâmetros e retorne a soma desses números. No `main`, peça ao usuário para inserir dois números e exiba a soma.

#### Requisitos:

- Implemente a função `soma`.
- Use passagem de parâmetros por valor.
- Exiba o resultado da soma no `main`.

#### Exemplo de Saída:

```
Digite o primeiro número: 5
Digite o segundo número: 3
A soma é: 8
```

### Exercício 2: Nível Médio

**Descrição:** Implemente um programa que tenha duas funções: `troca_valores` e `troca_valores_referencia`. A função `troca_valores` deve trocar os valores de

duas variáveis usando passagem por valor, e **troca\_valores\_referencia** deve fazer o mesmo usando passagem por referência. No **main**, demonstre o comportamento das duas funções e mostre como a passagem por referência realmente altera as variáveis originais.

#### Requisitos:

- Implemente as funções **troca\_valores** e **troca\_valores\_referencia**.
- Use passagem por valor em uma função e por referência na outra.
- Exiba os valores das variáveis antes e depois das chamadas de função no **main**.

#### Exemplo de Saída:

```
Valores antes da troca por valor: a = 5, b = 10
Valores após a troca por valor: a = 5, b = 10
Valores antes da troca por referência: a = 5, b = 10
Valores após a troca por referência: a = 10, b = 5
```

### Exercício 3: Nível Difícil

**Descrição:** Crie um projeto em C que seja dividido em três arquivos: **main.c**, **math\_operations.h** e **math\_operations.c**. No arquivo **math\_operations.h**, declare protótipos de funções para as operações de adição, subtração, multiplicação e divisão de dois números inteiros. Em **math\_operations.c**, implemente essas funções. No **main.c**, use essas funções para realizar operações matemáticas básicas em dois números fornecidos pelo usuário.

#### Requisitos:

- Crie e organize o projeto em três arquivos: **main.c**, **math\_operations.h**, **math\_operations.c**.
- Implemente as funções para adição, subtração, multiplicação e divisão.
- No **main**, peça ao usuário para inserir dois números e escolha uma operação para realizar (adicionar, subtrair, multiplicar ou dividir).
- Exiba o resultado da operação escolhida.

#### Exemplo de Saída:

```
Digite o primeiro número: 20
Digite o segundo número: 5
Escolha a operação (1 Adição, 2 Subtração, 3 Multiplicação, 4 Divisão): 4
O resultado da divisão é: 4
```